Adaptive Sequential Recommendation under Uncertain Scores

Coral Scharf
Technion – Israel Institute of Technology
Haifa, Israel
coralscharf@campus.technion.ac.il

Roee Shraga
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
rshraga@wpi.edu

Avigdor Gal
Technion – Israel Institute of Technology
Haifa, Israel
avigal@technion.ac.il

ABSTRACT

Top-K recommendation under uncertain scores addresses the problem of generating recommendation lists when item scores are modeled as uncertain rather than deterministic. Most existing approaches to recommendation under uncertainty follow a fixed-K static paradigm, in which the recommendation size is assumed to be known in advance and the entire list is produced in a single step. In this work, we propose a paradigm shift to an adaptive sequential setting. We study *adaptive sequential recommendation under uncertain scores*, in which items are recommended incrementally and future recommendations may depend on feedback observed during the interaction, thereby reducing uncertainty and affecting subsequent decisions. We propose an adaptive sequential recommendation model and present a framework for generating recommendations under uncertainty. We introduce sequential selection methods, derived from probabilistic ranking semantics, methods for updating the recommendation model based on observed feedback, and provably correct termination rules to determine when to halt the recommendation sequence. We empirically evaluate the proposed framework across multiple datasets, comparing adaptive sequential recommendation with the static top-K setting and studying the impact of feedback-based updating and adaptive termination on recommendation quality. Our results show that adaptive sequential recommendation improves recommendation quality over the static top-K approach across multiple settings, feedback-based updating contributes to recommendation effectiveness, and adaptive termination can determine the recommendation set size in a way that significantly improves recommendation quality.

PVLDB Reference Format:
Coral Scharf, Roee Shraga, and Avigdor Gal. Adaptive Sequential Recommendation under Uncertain Scores. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/coralscharf/AdaSeq_UncRec.

1 INTRODUCTION

Recommender systems commonly present users with a top-K answer, a ranked list of items sorted in descending order of relevance. Typically, this ranking is derived from scores computed based on user feedback, which may be collected in either implicit (*e.g.*, clicks, views) or explicit (*e.g.*, ratings) form [3, 23].

User feedback is noisy, often unreliable, and incomplete. Therefore, recommendations rely on inherently uncertain scores [6] and user–item pairs should be associated with a probability distribution (rather than deterministic) scores. To handle uncertain scores when generating recommendations, various probabilistic ranking semantics have been proposed [12, 32]. For example, the *Expected Score* semantics selects K items with the highest expected scores, whereas the *Global Top-K* semantics chooses the K items most likely to appear in the top-K positions across possible rankings.

In this work, we propose a paradigm shift from the fixed-K static paradigm, adopted by most existing approaches to recommendation under uncertainty [6, 24]. Using the fixed-K static paradigm, the system fixes a desired list size K in advance, producing the entire recommendation list in a single step and keeping it fixed throughout the interaction. While such an approach may work well in a deterministic setting, under uncertain scores the system has access only to a distribution over possible rankings rather than to a single fixed ranking. As a result, a static top-K answer can optimize recommendation quality only in expectation.

We propose an alternative, termed *adaptive sequential recommendation*, in which recommendations are provided *incrementally* rather than in a single bulk, and items are revealed to the user one by one (or in small batches). At each step, the system selects the next item(s) to be presented to the user, and may then incorporate the observed feedback into the recommendation model before choosing subsequent items. In this setting, the interaction may stop after only a few items, and the final number of recommended items is not necessarily known in advance.

Adaptive sequential recommendation under uncertainty carries the benefit of making use of user feedback that becomes available during the interaction. User reaction (*e.g.*, by providing a rating or an implicit signal), may reveal additional information about user's preferences. Such feedback can reduce uncertainty, as was demonstrated in areas such as crowdsourced query processing [4], query-guided uncertainty resolution [7], and schema matching [26, 33]. It may improve the stochastic understanding of user preferences and, consequently, the choice of future recommendations. As the recommendation process unfolds, feedback accumulates, and the usefulness of each item depends not only on its own relevance but also on the items shown earlier and the outcomes observed for them. The system can therefore exploit accumulated feedback to update its belief about the user's preferences and item relevance, and steer subsequent recommendations to improve the overall expected quality of the delivered recommendation sequence. This sequential adaptivity naturally induces an exploration–exploitation trade-off, where early items can be used to reduce uncertainty, while later

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

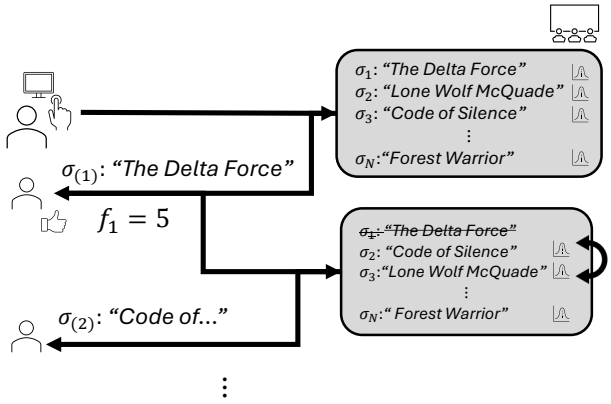


Figure 1: Motivating example illustration.

U_1	**Score distribution** $S(\sigma)$	**Rank distribution** $R(\sigma)$
σ_1	$\{(1, 0.45), (5, 0.55)\}$	$\{$(1st, 0.55), (2nd, 0.18), (3rd, 0.27)$\}$
σ_2	$\{(0.5, 0.4), (4.5, 0.6)\}$	$\{(1st, 0.27), (2nd, 0.33), (3rd, 0.40)\}$
σ_3	$\{(3, 0.6), (4, 0.4)\}$	$\{$(1st, 0.18), (2nd, 0.49), (3rd, 0.33)$\}$

⇓ recommend σ_1, observe feedback $f_1 = 5$

U_2	**Score distribution** $S(\sigma)$	**Rank distribution** $R(\sigma)$
~~σ_1~~	~~$\{(1, 0), (5, 1)\}$~~	~~1st~~
σ_2	$\{(0.5, 0.4), (4.5, 0.6)\}$	$\{(1st, 0), (2nd, 0.6), (3rd, 0.4)\}$
σ_3	$\{(3, 0.6), (4, 0.4)\}$	$\{(1st, 0), (2nd, 0.4), (3rd, 0.6)\}$

Table 1: Motivating example. Initial score and rank distributions in U_1 (top), and the updated distributions in U_2 after recommending σ_1 and observing feedback (bottom).

items leverage the reduced uncertainty to increase expected quality. During the exploitation phase, the system has the freedom to determine whether continuing the recommendation sequence is worthwhile, since additional items may not always improve the expected quality of the final result.

Example 1. *To illustrate the benefit of adaptive sequential recommendation under uncertain scores, consider a scenario where movies are recommended to users (see Figure 1 and Table 1 for illustration). As in a typical setting, a recommender system aims to provide a user (left-hand side of Figure 1) top-K recommendation out of a set of possible options (right-hand side of Figure 1). However, in this case, each of those option comes with a score distribution representing the* ***uncertainty*** *of its score. The system* ***sequentially*** *provides recommendation (in the illustration the movie "The Delta Force") and gets a deterministic score from the user (in the illustration* 5*). Then, the system* ***adapts*** *(illustrated as an example with the double-sided arrows), continuing with a new recommendation.*

*To zoom in, consider the three candidate movies with initial score and rank distributions, as given in the top part of Table 1. The initial score distribution was generated using an uncertainty-aware recommendation model that leverages users' historical feedback (*e.g.*, [31]), and the rank distribution was induced from it over the possible worlds, using an algorithm such as RankDist [24]. Under the static top-K approach, and using the Global Top-K semantics with $K = 2$, the result consists of the two movies with the highest probability of appearing in the top-2 positions, namely σ_1 and σ_3, with accumulated probabilities over ranks* 1 *and* 2 *of* 0.73 *and* 0.67*, respectively (see probabilities marked in red at the top of Table 1).*

With adaptive sequential recommendation, assume movies are selected one at a time. Assume that the system first presents to the user the item with the highest probability of being ranked first, namely σ_1 (with probability 0.55 *of occupying the first rank). Then, suppose that the user provides feedback by assigning it a score of* 5 *(other valid observed scores are also possible, but we use this outcome for illustration). As a result, the uncertainty regarding the score of σ_1 is resolved, as shown in the bottom part of Table 1, and assuming we do not want to recommend the same item twice, this item is no longer a candidate for recommendation. Conditioned on this feedback, the score distributions of σ_2 and σ_3 remain unchanged, but their rank distributions are updated. In particular, while initially σ_3 was more likely to occupy the second rank, after conditioning on the observed feedback, σ_2 becomes more likely to be ranked second (with probability* 0.6*, compared to* 0.4 *for σ_3, see probabilities marked in blue at the bottom of Table 1), and is therefore selected next. Thus, while the static top-*2 *answer is $[\sigma_1, \sigma_3]$, the adaptive sequential process recommends σ_1 first and then σ_2, showing how feedback observed during the interaction may change subsequent recommendation decisions.*

In this paper, we study recommendation under uncertain scores through the lens of *adaptive sequential recommendation*. We begin with preliminaries on ranking in recommender systems under uncertain scores (Section 2). We then present the adaptive sequential recommendation model and formally define the problem (Section 3). In Section 4, we describe the adaptive sequential recommendation pipeline and present methods for generating recommendations under uncertainty, including adaptive sequential selection semantics, recommendation model updates, and termination rules. Our empirical evaluation is presented in Section 5, where we compare adaptive sequential recommendation to the static top-K setting, analyze the effect of feedback on recommendation quality throughout the recommendation sequence, and evaluate the effectiveness of the adaptive termination rules. We review related work (Section 6) and conclude in Section 7. Our main contributions are:

- We introduce the problem of *adaptive sequential recommendation* under uncertain scores, and offer an adaptive sequential recommendation model in which items are recommended incrementally and future decisions may depend on feedback observed during the interaction.
- We present an adaptive sequential recommendation framework, with sequential recommendation generation based on probabilistic ranking semantics an update protocol of the recommendation model based on observed feedback.
- We offer an optimal (by expectation) termination rule for deciding whether to continue the recommendation sequence.
- We empirically evaluate the proposed framework, comparing adaptive sequential recommendation with the static top-K setting, analyzing the effect of feedback throughout the recommendation sequence, and examining the ability

to determine the recommendation size dynamically. Our results show that the adaptive approach improves the quality of the recommendations over static methods and that the size of the recommendations can be effectively determined during the adaptive process.

2 PRELIMINARIES: RECOMMENDER SYSTEMS

Let U_I and U_U denote universes of M items and Q users, respectively, where $p_i \in U_I$ indicates a single item and $u_j \in U_U$ represents a single user. In this paper, we consider an explicit-feedback setting [14, 25], where each user assigns a score s to items they interact with, chosen from a finite set *Scores* (e.g., $\{1, 2, 3, 4, 5\}$). Each interaction consists of a user identifier (u_j), an item identifier (p_i), and a corresponding explicit score s. From this dataset D, we can extract a rating matrix R, where each row corresponds to a user, each column corresponds to an item, and each entry represents the score assigned by the user to the item.

User-provided scores reflect subjective preferences that may be inconsistent, biased, or imprecise [6]. To capture this inherent uncertainty of explicit feedback, a pair (u_j, p_i) is associated with a score distribution $S(p_i, u_j)$. This distribution represents the likelihood that the user would assign each possible score to the item, thereby capturing the uncertainty present in explicit feedback. $S(p_i, u_j)$ is a random variable, and $s(p_i, u_j)$ denotes a possible realization of this variable, with probability of $\Pr(S(p_i, u_j) = s(p_i, u_j))$. We assume that each $S(\cdot)$ has a finite set of possible values, and that $s(\cdot)$ is one of these values. If continuous score distributions are used, they can be discretized, for example by binning into intervals. For a fixed user u_j, let $U \subseteq U_I$ denote the candidate set of N items considered for recommendation. We assume that $S(\sigma)$ and $S(\sigma')$ are pair-wise probabilistically independent for all $\sigma, \sigma' \in U$ [27]. For brevity, we denote a user-item pair (u_j, p_i) by a tuple σ. When focusing on a recommendation task for a fixed user, σ can be viewed simply as an item associated with that user.

2.1 Ranking with Uncertain Scores

Uncertain scores induce *possible worlds*, each representing a deterministic realization and is associated with a probability indicating how likely it is to reflect the true data [29]. In each world, each tuple $\sigma \in U$ independently samples a score from its distribution $S(\sigma)$, and the combination of these sampled values across all tuples in U defines a possible world. Let W denote the set of all possible worlds. For a world $w \in W$, we use $s_w(\sigma)$ to denote the deterministic score assigned to item σ in that world. The probability of a particular world occurring is given by the product of the probabilities of the sampled scores across all items, given by $Pr(w) = \prod_{\sigma \in U} Pr(s_w(\sigma))$.

Each world induces a deterministic ranking over the items. We let $r_w(\sigma)$ represent the rank of item σ in world w, where a lower value indicates higher rank. The distribution over ranks for σ, denoted $R(\sigma)$, is defined by the aggregation of its positions across all possible worlds. That is, for $1 \le i \le N$, $Pr\,(R(\sigma) = i) = \sum_{w \in W} Pr(w) \cdot \delta\,(r_w(\sigma) = i)$, where δ is an indicator function, returning 1 if $r_w(\sigma) = i$ and 0 otherwise.

For some user u_j and some constant $K \ge 1$, a candidate set $U \subseteq U_I$ of N tuples (items) (referring to each item in U as a tuple σ) can be ranked in a decreasing order of attractiveness to u_j. Each tuple $\sigma \in U$ is associated with a score distribution $S(\sigma)$ and the induced rank distribution $R(\sigma)$. We define $A = \{\sigma_1, \ldots, \sigma_K\} \subseteq U$ to be a top-K answer, which may be ordered or unordered, and denote by $\sigma_{(i)}$ the tuple appearing in the i-th position of A, if ordering is defined. Usually, ranking approaches follow a no-repetition assumption, namely, recommended items are distinct and no item is recommended more than once. When ranking with uncertain scores, multiple approaches can be used to generate A. For illustration, we provide next two approaches, the first is score-based and the second rank-based [12]. An analysis of other approaches for ranking with uncertain scores is available in [24].

Expected Score: tuples with highest expected score, computed by $\sum_{s \in S(\sigma)} Pr(S(\sigma) = s) \cdot s$ for a discrete distribution.

Global Top-k: tuples with highest probability of being ranked within the range of ranks from 1 to K, $Pr\,(R\,(\sigma) \le K)$.

Example 2. *To illustrate the notation and semantics, recall the example in Table 1. In the initial universe U_1, item σ_1 has score distribution $S(\sigma_1) = \{(1, 0.45), (5, 0.55)\}$, meaning that its score is 1 with probability 0.45 and 5 with probability 0.55. Similarly, $S(\sigma_2) = \{(0.5, 0.4), (4.5, 0.6)\}$ and $S(\sigma_3) = \{(3, 0.6), (4, 0.4)\}$. From these score distributions, one can derive the rank distributions. For example, σ_1 is ranked first with probability 0.55, second with probability 0.18, and third with probability 0.27.*

Assume that the goal is to identify the first item to recommend, we next illustrate the result under each semantics. Under the Expected Score semantics, the expected scores are 3.2 for σ_1 ($1 \cdot 0.45 + 5 \cdot 0.55$), 2.9 for σ_2 ($0.5 \cdot 0.4 + 4.5 \cdot 0.6$), and 3.4 for σ_3 ($3 \cdot 0.6 + 4 \cdot 0.4$). Hence, Expected score returns σ_3 as the top-1 answer. In contrast, under Global top-1, each tuple is ranked according to its probability of appearing in the first position, namely 0.55, 0.27, and 0.18 for σ_1, σ_2, and σ_3, respectively. Thus, Global Top-1 returns σ_1. This example illustrates that different semantics for ranking under uncertain scores may produce different results.

2.2 Measuring Precision under Uncertain Scores

To quantify the effectiveness of a recommendation, we define a user-specific *reference* list $A' \subseteq U_I$, which can be conceived as the items most relevant to the user, given the opportunity to explore all items in U. We follow the conservative reference definition introduced in [24], in which A' contains the top-K highest ranked items from U, selected according to their ground-truth scores.

In this paper, we focus on *P@K*, leaving other quality measures to future work. P@K is a rank-based metric that measures how many relevant items are included in the top-K results, ignoring their specific positions. Given a top-K answer A and a reference list A', *P@K of A with respect to A'* is given by

$$\text{P@}K\,(A, A') = \frac{1}{K} \cdot |A \cap A'| \tag{1}$$

where $P@K\,(A, A')$ denotes the precision value at K of a recommendation answer A with respect to the reference set A'. For brevity, we also write this as $P@K_A$.

For probabilistic ranking, we assess answer quality using the expected value of the measure, where a higher expectation indicates a higher quality answer. Since uncertain scores induce multiple possible worlds, the deterministic top-K answer may vary across worlds. Accordingly, for each possible world $w \in W$, let A_w^K denote

the deterministic top-K answer induced by the realized scores in w. We next define the expected precision of A.

The *expected P@K of A* is given by:

$$E\left[\mathrm{P@}K_A\right] = \sum_{w \in W} Pr\left(w\right) \cdot \mathrm{P@}K\left(A, A_w^K\right) \tag{2}$$

That is, the expectation is taken over all possible worlds induced by the uncertain scores.

Equivalently, using the rank distribution derived in [24], it can be computed as:

$$E\left[\mathrm{P@}K_A\right] = \frac{1}{K} \cdot \sum_{\sigma \in A} Pr\left(R(\sigma) \leq K\right) \tag{3}$$

3 MODEL AND PROBLEM DEFINITION

Using the notation in Section 2, we now provide a model for adaptive sequential recommendations. For a user $u_j \in U_U$, the recommendation process starts from an initial universe $U_1 \subseteq U_I$ of N items (tuples) to be ranked for the user. Each tuple $\sigma \in U_1$ is associated with a score distribution $S(\sigma)$ over the score domain *Scores*, and with the induced rank distribution $R(\sigma)$.

An adaptive sequential recommendation process generates an ordered answer $A = (\sigma_{(1)}, \ldots, \sigma_{(K)})$ consisting of items from U_1, by selecting one item at each step. We denote by $\sigma_{(t)}$ the tuple selected at step t, which appears in the t-th position of A. For simplicity, we present the model for a setting where a single item is selected at a time, noting that it can be extended to recommending a batch of items in each iteration. The final recommendation length is denoted by K.

After recommending $\sigma_{(t)}$ at step t, the system may observe user feedback on the recommended item. In this work, we assume that feedback is given as an explicit score from the same domain *Scores*, and we denote it by $f_t \in Scores$. We denote by $h_t = \langle \sigma_{(t)}, f_t \rangle$ the observation recorded at time t, consisting of the recommended item $\sigma_{(t)}$ and, when available, the observed feedback f_t ($f_t = \emptyset$ in the absence of explicit feedback). The interaction history up to step K' is the sequence $H_{K'} = \langle h_t \rangle_{t=1}^{K'}$ (and in particular, $H = H_K$).

We allow the current universe, as well as the score and rank distributions of the items, to change at step t as a function of the observed history H_{t-1}. Let $U_t \subseteq U_1$ denote the universe considered at step t, with respect to which the current score and rank distributions are defined. In addition, we denote by $S(\sigma \mid H_{t-1})$ the score distribution of tuple $\sigma \in U_t$ conditioned on the feedback observed so far in the first $t-1$ steps. The corresponding conditioned rank distribution is denoted by $R(\sigma \mid H_{t-1})$, and is induced by the conditioned score distributions of the tuples in U_t. Formally, $R(\sigma \mid H_{t-1})$ is defined as the aggregation of positions of σ over the possible worlds induced by $\{S(\cdot \mid H_{t-1})\}$. As shorthand, we write $S_t(\sigma) = S(\sigma \mid H_{t-1})$ and $R_t(\sigma) = R(\sigma \mid H_{t-1})$.

Example 3. *Recall the example presented in Table 1. After recommending the first item $\sigma_{(1)} = \sigma_1$, the user provides feedback $f_1 = 5$. Following this interaction, the observation $h_1 = \langle \sigma_{(1)}, f_1 = 5 \rangle$ is recorded, and the interaction history up to step 1 is $H_1 = \langle h_1 \rangle$. This history may update the universe U_1 to U_2, as well as the score and rank distributions of the items. Consider σ_2 for example. While its score distribution does not change, namely $S_1(\sigma_2) = S_2(\sigma_2) = \{(0.5, 0.4), (4.5, 0.6)\}$, its rank distribution changes from $R_1(\sigma_2) = \{(1\text{st}, 0.27), (2\text{nd}, 0.33), (3\text{rd}, 0.4)\}$ to $R_2(\sigma_2) = \{(1\text{st}, 0), (2\text{nd}, 0.6), (3\text{rd}, 0.4)\}$.*

Given the history observed up to step $t-1$, the expected value of the metric is conditioned on the feedback observed so far. We evaluate the quality of the current prefix $A_t = (\sigma_{(1)}, \ldots, \sigma_{(t)})$ with respect to the initial universe U_1, to account for previously recommended items that were possibly removed from the current universe, as well as any feedback observed on them.

To this end, let $W(H_{t-1})$ denote the set of possible worlds over U_1 that are consistent with the observed history H_{t-1}, namely, the worlds induced by the initial universe and score distributions in which every tuple for which feedback was observed has the score revealed by the user. Formally,

$$\begin{aligned} W(H_{t-1}) = \{w \in W \mid s_w(\sigma_{(i)}) = f_i \text{ for every } i < t \\ \text{such that } h_i = \langle \sigma_{(i)}, f_i \rangle \text{ and } f_i \neq \emptyset\} \end{aligned} \tag{4}$$

Given a recommendation prefix $A_t = (\sigma_{(1)}, \ldots, \sigma_{(t)})$, we define its conditional expected precision at step t with respect to the observed history H_{t-1} by:

$$E\left[\mathrm{P@}t_{A_t} \mid H_{t-1}\right] = \sum_{w \in W(H_{t-1})} Pr(w \mid H_{t-1}) \cdot \mathrm{P@}t\left(A_t, A_w^t\right) \tag{5}$$

where A_w^t denotes the deterministic top-t answer induced by w over the initial universe U_1.

The conditional expected precision can also be computed using a conditional evaluation rank distribution. It is noteworthy that the conditional evaluation rank distribution, denoted by $R_t^{eval}(\sigma)$, is defined with respect to the initial universe U_1 under the conditioned possible worlds $W(H_{t-1})$. The score distributions used to compute this conditional evaluation rank distribution are the initial score distributions, in which the score distribution of each item for which the user provided feedback is replaced by a deterministic distribution at the feedback value. Using this notation, the conditional expected precision can be written equivalently as follows:

$$E\left[\mathrm{P@}t_{A_t} \mid H_{t-1}\right] = \frac{1}{t} \cdot \sum_{\sigma \in A_t} Pr\left(R_t^{eval}(\sigma) \leq t\right) \tag{6}$$

Given a user u_j, an initial universe U_1, score distributions $S(\sigma)$ for each $\sigma \in U_1$ (inducing rank distributions $R(\sigma)$), and a target quality measure, the goal is to generate an ordered recommendation answer A sequentially so as to optimize its quality, where evaluation at each step is performed under the conditioned model induced by the observed feedback history.

Problem 1 (Adaptive Sequential Recommendation under Uncertain Scores). *Let u_j be a user, U_1 an initial universe with score distributions $S(\sigma)$ for each $\sigma \in U_1$ (inducing rank distributions $R(\sigma)$), and M a quality measure.*

The problem of adaptive sequential recommendation under uncertain scores *(ASRU for short) is to generate an answer $A = (\sigma_{(1)}, \ldots, \sigma_{(K)})$ of some length K, with the goal of optimizing the quality measure M.*

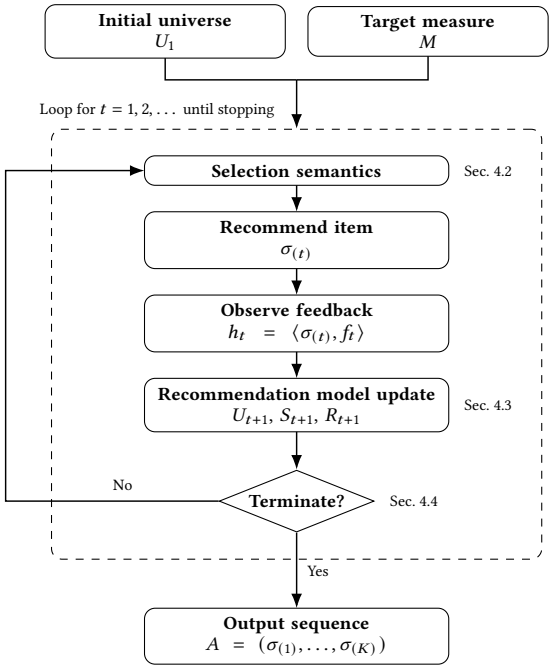


Figure 2: Adaptive sequential recommendation under uncertain scores (ASRU) pipeline.

4 UNCERTAIN ADAPTIVE SEQUENTIAL RECOMMENDATION

In this section, we present an adaptive sequential recommendation framework under uncertain scores, that aims at solving Problem 1. We begin with an overview of the ASRU pipeline (Section 4.1). We then describe the three main components of the framework, namely, *selection semantics* (Section 4.2), *recommendation model update* (Section 4.3), and *termination rule* (Section 4.4) components.

4.1 ASRU Pipeline

We now describe a pipeline for ASRU (see Figure 2 for illustration).

For a user $u_j \in U_U$, the pipeline begins by specifying an initial universe of candidate items $U_1 \subseteq U_I$ to be considered for recommendation, together with score distributions $S(\sigma)$ for all $\sigma \in U_1$. This universe serves as the initial universe for the adaptive process. It is typically produced by an uncertainty-aware recommendation model that generates score distributions (*e.g.*, using Wang et al. [31]), together with a first-stage candidate generation method that may filter the possibly large set of candidate items. The system also selects a quality measure M that defines the recommendation objective under uncertain scores, such as $P@K$ (Section 2).

Given these inputs, recommendations are generated through an adaptive sequential process that iterates over selection semantics, item recommendation, feedback observation, and model update until a termination condition is met, yielding a possibly data-dependent final length K. At each step t, *selection semantics* chooses the next item to present from the current universe set U_t. Different selection semantics, derived from probabilistic ranking semantics, may be used for this step (as will be described in Section 4.2).

After an item is recommended, the system may *observe feedback* from the user, which may be used to *update the recommendation model*, modifying the universe and potentially the associated score and induced rank distributions before the next recommendation step (Section 4.3). The system then decides whether to continue or stop (Section 4.4). If the termination condition is not satisfied, the process proceeds with an updated recommendation model. Otherwise, it outputs the resulting recommendation sequence.

Overall, ASRU generates recommendations step by step, allowing feedback observed during the interaction to affect subsequent selections and possibly also the final recommendation length. We next discuss in details sequential recommendation generation, feedback incorporation, and termination.

4.2 Sequential Selection Semantics

We now describe how to generate the next recommendation at step t given the current recommendation model. Recall that at step t the system has conditional score distributions $S_t(\sigma)$ and the induced rank distributions $R_t(\sigma)$ with respect to the current universe U_t (Section 3). Thus, the semantics applied at step t depend on the current recommendation model, namely, the universe U_t, and the conditional score and rank distributions.

A sequential selection semantics specifies how to select the next tuple $\sigma_{(t)}$ from universe U_t based on $S_t(\cdot)$ and $R_t(\cdot)$. We introduce three adaptations of static ranking semantics (Section 2), which do not commit in advance to a fixed final recommendation length, naturally support variable-length outputs.

- **Adaptive Expected Score (A-ES)** recommends at step t the tuple in U_t with the highest conditional expected score under $S_t(\sigma)$, namely, $\sigma_{(t)} = \text{argmax}_{\sigma \in U_t} E[S_t(\sigma)]$.
- **Most Probable at Position** 1 **(MPP-1)** recommends at step t the tuple in U_t with the highest probability of being ranked first, namely, $\sigma_{(t)} = \text{argmax}_{\sigma \in U_t} \Pr(R_t(\sigma) = 1)$.
- **Adaptive Global Top-t (A-GT-t)** recommends at step t the tuple in U_t that maximizes the probability of being ranked within the top-t positions, namely, $\sigma_{(t)} = \text{argmax}_{\sigma \in U_t} \Pr(R_t(\sigma) \leq t)$. Intuitively, it selects items that are most likely to belong to the prefix being constructed.

4.3 Recommendation Model Update

After step t, the system records the observation h_t (Section 3), which may include explicit feedback f_t on the recommended item $\sigma_{(t)}$. Then, the system updates the recommendation model for step $t+1$.

In our setting, once $\sigma_{(t)}$ has been recommended, it is excluded from future consideration (under the no-repetition assumption). Accordingly, the universe at step $t+1$ is $U_{t+1} = U_t \setminus \{\sigma_{(t)}\}$.

Feedback observation f_t may provide evidence about the user's preferences that is relevant to other tuples in U_{t+1}. Therefore, it affects subsequent recommendations through the score distributions $S_{t+1}(\cdot)$ over tuples in U_{t+1}, which in turn induce updated rank distributions $R_{t+1}(\cdot)$ with respect to the reduced universe.

The system forms the updated recommendation model for step $t+1$, namely, the universe U_{t+1}, the score distributions $S_{t+1}(\cdot)$, and the induced rank distributions $R_{t+1}(\cdot)$. The next recommendation step then selects $\sigma_{(t+1)}$ by applying a selection semantics over U_{t+1} with respect to this updated model. Thus, the update mechanism

determines the updated recommendation model on which the sequential selection semantics of Section 4.2 operate in step $t+1$.

We consider three alternatives for score distribution update.

4.3.1 No score update. The feedback does not modify the score distributions. Formally, for every tuple $\sigma \in U_{t+1}$, we keep $S_{t+1}(\sigma) = S_t(\sigma)$. Although the score distributions of the remaining tuples are unchanged, the induced rank distributions are recomputed with respect to the reduced universe U_{t+1}.

4.3.2 Local and global updates. Score update involves the impact observation $h_t = \langle \sigma_{(t)}, f_t \rangle$ has on the score distributions of other tuples in U_{t+1}. We assume an embedding-based recommendation model, with a user embedding for each user and an item embedding for each tuple. In addition, we assume that, for each user-item pair, these embeddings induce a score distribution $S_t(\sigma)$ derived from an underlying normal distribution.

To guide the update, we define the mean feedback error

$$e_t^{\mu} = f_t - E\left[S_t(\sigma_{(t)})\right] = f_t - \mu_t(\sigma_{(t)}) \tag{7}$$

which quantifies the discrepancy between the observed feedback and the predicted mean score of the recommended item.

Local score update. The local score update modifies the score distributions only of tuples that are similar to the recommended item $\sigma_{(t)}$ in an item-embedding space. Let $v(\sigma) \in \mathbb{R}^{dim}$ denote the embedding of tuple σ. Let $NN_L(\sigma_{(t)}) \subseteq U_{t+1}$ denote the set of the L nearest neighbors of $\sigma_{(t)}$ in the updated universe, and let $sim(\sigma, \sigma_{(t)}) \in [0,1]$ denote the similarity between σ and $\sigma_{(t)}$ (e.g., cosine similarity).

This update is based on the assumption that feedback on $\sigma_{(t)}$ may also carry information about tuples that are close to it in the embedding space, with the strength of the propagated effect determined by their similarity. Accordingly, for every neighbor $\sigma \in NN_L(\sigma_{(t)})$, we update the mean parameter by

$$\mu_{t+1}(\sigma) = \mu_t(\sigma) + \alpha \cdot sim(\sigma, \sigma_{(t)}) \cdot e_t^{\mu} \tag{8}$$

where $\alpha > 0$ controls the strength of the propagation. For tuples outside the neighbor set, the mean remains unchanged. The updated score distribution $S_{t+1}(\sigma)$ is then obtained from the updated underlying distribution with mean $\mu_{t+1}(\sigma)$. Thus, the observed feedback is propagated locally, only to tuples that are close to $\sigma_{(t)}$ in the embedding space.

Global score update. The global score update modifies the user representation itself based on the observed feedback, while keeping the item representations fixed. Let $z_t(u_j) \in \mathbb{R}^{dim}$ denote the embedding of user u_j at step t, and let $v(\sigma) \in \mathbb{R}^{dim}$ denote the embedding of tuple σ. We assume that, at step t, the parameters of the score distribution $S_t(\sigma)$ are generated from the pair $(z_t(u_j), v(\sigma))$ for the current user u_j and each $\sigma \in U_t$.

After observing $h_t = \langle \sigma_{(t)}, f_t \rangle$, we update the user embedding:

$$z_{t+1}(u_j) = z_t(u_j) + \alpha \cdot e_t^{\mu} \cdot v(\sigma_{(t)}) \tag{9}$$

where $\alpha > 0$ is an update parameter. This update may be viewed as a single correction step that adjusts the user representation based on the prediction error on the observed item.

The updated user embedding is then used to regenerate the score distributions of all tuples in the updated universe, thereby yielding

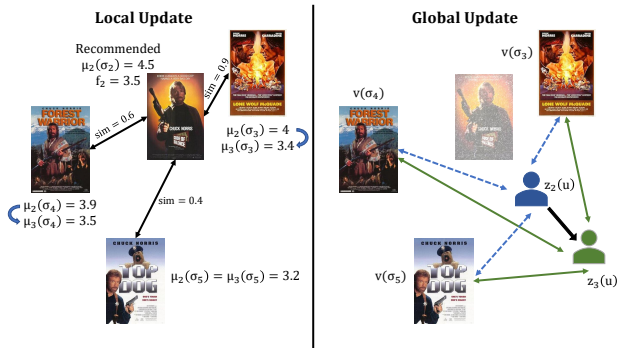


Figure 3: Update example illustration.

$S_{t+1}(\sigma)$ for every $\sigma \in U_{t+1}$. Unlike the local update, which affects only tuples in the vicinity of $\sigma_{(t)}$, the global update may affect all tuples through the shared user representation.

4.3.3 Score update impact. Score updates may affect the recommendations returned by the adaptive semantics differently. Score-based semantics that rely only on the current score distributions, such as A-ES, are unaffected under the no score update alternative, since the score distributions of the remaining tuples are unchanged. By contrast, both the local and global score updates may affect A-ES, since they modify the score distributions of tuples in U_{t+1}. Rank-based semantics that rely on the induced rank distributions, such as MPP-1 and A-GT-t, may be affected even under no score update, since removing $\sigma_{(t)}$ changes the universe with respect to which rank distributions are computed. Local and global score updates may further affect such semantics through the resulting changes in the score distributions of tuples in U_{t+1}.

Example 4. *To illustrate the recommendation model update alternatives, consider Example 1 and assume that after two steps the last recommended movie was "Code of Silence" ($\sigma_{(2)}$) and the user provided a feedback score of* 3.5. *Moving to the third recommendation, assume that the universe U_3 contains three candidate movies: "Lone Wolf McQuade" (σ_3), "Forest Warrior" (σ_4), and "Top Dog" (σ_5). Since $\sigma_{(2)}$ has already been recommended, it is excluded from future consideration and therefore does not belong to U_3. Based on the second recommendation and the observed feedback, the system now chooses how to update the remaining tuples' score distributions.*

Under the no score update alternative, the score distributions of the remaining tuples are left unchanged, and only the rank distributions are recomputed with respect to the reduced universe U_3.

Figure 3 illustrates the two score update alternatives. In the local update (Figure 3(left)), item embeddings are used only to identify tuples that are most similar to the last recommended tuple and to determine the strength of the update. Assume that the similarities of σ_3, σ_4, and σ_5 to $\sigma_{(2)}$ are 0.9, 0.6, *and* 0.4, *respectively. Further assume that before receiving the feedback, the expected score of $\sigma_{(2)}$ was* 4.5, *yielding an error of $e_2 = 3.5 - 4.5 = -1$. For a local update with the two nearest neighbors, the means of σ_3 and σ_4 are shifted downward according to Eq. 8 (with $\alpha = 0.667$), reflecting that the observed feedback was lower than expected. The mean of σ_5 does not change, since it is not among the two nearest neighbors of $\sigma_{(2)}$. Such*

an update can change the relative order of the next two candidate items in terms of their means, affecting subsequent recommendations.

In the global update (Figure 3(right)), the feedback is used to update the user representation, and then the means of all remaining tuples in U_3 are recomputed with respect to the updated user representation. As a result, all tuples in the universe may be affected, not only the nearest neighbors of $\sigma_{(2)}$. This allows the feedback on a single recommendation to induce a broader change in the recommendation model, potentially changing the relative order of all remaining tuples. The dashed blue connections of Figure 3(right) represent the interactions between the user embedding and the item embeddings at step 2, which determine the score means before the update. The green connections represent the corresponding interactions at step 3, after incorporating the observed feedback, and therefore determine the updated means.

4.4 Early Termination

At the end of each recommendation step, the system determines whether to continue to another recommendation step or halt. One option is to stop after producing a fixed number of recommendations. Alternatively, the termination decision may be taken adaptively during the interaction. We therefore analyze whether extending the current prefix by one tuple improves the expected recommendation quality, based on the feedback observed so far.

Let $H_{t-1} = \langle h_1, \ldots, h_{t-1} \rangle$ denote the interaction history observed after the first $t-1$ recommendation steps, and let $A_{t-1} = (\sigma_{(1)}, \ldots, \sigma_{(t-1)})$ be the current recommendation prefix (as described in Section 3). Assume that $\sigma_{(t)}$ is the next candidate tuple, and let $A_t = (\sigma_{(1)}, \ldots, \sigma_{(t)})$ denote the result of appending $\sigma_{(t)}$ to A_{t-1}. For simplicity, we consider extension by a single tuple, while the analysis can be adapted to a batch of items. Our goal is to *build* answers sequentially in a way that improves the evaluation metric value. To this end, we derive a condition under which extending an answer from A_{t-1} to A_t increases its quality.

We focus on the conditional expected precision (Section 3). Throughout the analysis, $R_t(\cdot)$ denotes the conditioned evaluation rank distribution, namely, the rank distribution defined with respect to the initial universe U_1 under the observed history H_{t-1}. Using this notation, the conditional expected precision is computed by Eq. 6, with respect to the conditional evaluation rank distribution over the initial universe.

Proposition 1 characterizes when extending the current prefix by one tuple improves the conditional expected precision.

Proposition 1. *Let A_{t-1} be the current recommendation prefix after observing history H_{t-1}, and let A_t be the result of appending a candidate tuple $\sigma_{(t)}$ to A_{t-1}. Then*

$$E\left[P@t_{A_t} \mid H_{t-1}\right] > E\left[P@(t-1)_{A_{t-1}} \mid H_{t-1}\right] \tag{10}$$

if and only if

$$\sum_{\sigma \in A_{t-1}} Pr\left(R_t(\sigma) = t\right) \; + \; Pr\left(R_t(\sigma_{(t)}) \leq t\right) > E\left[P@(t-1)_{A_{t-1}} \mid H_{t-1}\right] \tag{11}$$

Proof Sketch. Using Eq. 6, we compare the conditional expected precision of A_{t-1} to that of A_t (Eq. 10). Since A_t extends A_{t-1} by adding $\sigma_{(t)}$, we decompose the conditional expected precision of A_t into the contribution of the current prefix A_{t-1} over the first $t-1$ positions, the probability mass associated with tuples in A_{t-1} occupying rank t, and the contribution of the new tuple $\sigma_{(t)}$. Substituting this decomposition and simplifying the resulting inequality yields Eq. 11. The complete proof appears in the technical report.[1] □

Proposition 1 provides a termination criterion for adaptive sequential recommendation under expected precision. It compares the expected contribution of extending the current prefix by $\sigma_{(t)}$ against the expected precision of the current prefix itself. Accordingly, the recommendation process should continue only when Eq. 11 holds. That is, extending the current prefix is beneficial only when the contribution of the new tuple, together with the probability mass by which previously selected tuples now occupy rank t, exceeds the expected precision of the current prefix.

We also consider a *marginal-item* termination variant, motivated by the desire to isolate the benefit of the candidate tuple itself when extending the list from A_{t-1} to A_t. In particular, when moving from P@$(t-1)$ to P@t, the precision reference in each possible world changes from the top-$(t-1)$ tuples to the top-t tuples. As a result, previously selected tuples may also benefit from the extension, since they may now additionally contribute through the event $R_t(\sigma) = t$. Under this variant, extending the prefix is beneficial iff

$$Pr\left(R_t(\sigma_{(t)}) \leq t\right) > \frac{1}{t-1} \cdot \sum_{\sigma \in A_{t-1}} Pr\left(R_t(\sigma) \leq t-1\right) \tag{12}$$

That is, we compare only the probability that the newly added tuple belongs to the top-t against the conditional expected precision of the current prefix. We note that this condition is a sufficient condition for satisfying Eq. 11.

5 EMPIRICAL EVALUATION

We present next an empirically evaluation of the proposed ASRU pipeline (Section 4) as a solution to Problem 1. Our evaluation compares adaptive sequential recommendation to static recommendation, investigates the impact of the adaptive update methods, and examines the ability to determine the recommendation size dynamically.

The main findings of this section are:

- **Better Answer Sets (Section 5.2):** Adaptive sequential recommendation can generate better answer sets $A = (\sigma_{(1)}, \ldots, \sigma_{(K)})$ than static recommendation, in terms of $P@K$. Among the examined adaptive semantics, A-GT-t achieves the best overall performance, outperforming both the other adaptive semantics and the static Global Top-k baseline. Moreover, the advantage of A-GT-t over Global Top-k becomes more pronounced in the lower parts of the recommendation list, once exploration is replaced with exploitation.
- **Improved Recommendation Quality (Section 5.3):** Universe and score updating are effective in improving recommendation quality. In particular, local score update achieves the best overall results among the examined update options, and its contribution becomes more substantial when sufficiently many feedback items are observed during the interaction.

[1] https://github.com/coralscharf/AdaSeq_UncRec

- **Effective Early Termination (Section 5.4):** Dynamically determining the recommendation size using the marginal-item termination rule improves recommendation quality over fixed-size lists by a large margin. The marginal-item termination rule by-and-large generates a distribution of list sizes that match user preferences.

The remainder of this section is organized as follows. Section 5.1 describes the experimental setup. In Section 5.2, we analyze adaptive sequential *vs.* static recommendations, and in Section 5.3 we analyze the effect of different adaptive update methods. Section 5.4 evaluates the termination mechanism to dynamically determine the size of the recommendation. Section 5.5 discusses the computational overhead of the process.

5.1 Experimental Methodology

The framework is implemented in Python, and is publicly available.[2] Experiments were conducted on a server with 2 NVIDIA RTX 5000 Ada Generation GPUs, 250 GB RAM, and Rocky Linux 9.5.

5.1.1 Datasets: We conduct experiments on three real-world datasets that are common benchmarks in recommender systems.

- **ml-25m** [6] was collected from the MovieLens website, with 24,945,870 ratings of 162,541 users on 32,720 movies. Rating values are in $\{0.5, 1.0, 1.5, \ldots 5.0\}$.
- **Netflix** [6] was collected from the Netflix website, with 100,461,928 ratings of 472,987 users on 17,769 movies. Rating values are in $\{1.0, 2.0, 3.0, 4.0, 5.0\}$.
- **Amazon-Book** [19] was collected from Amazon, with 4,747,707 ratings of 127,407 users on 23,931 books. Rating values are in $\{1.0, 2.0, 3.0, 4.0, 5.0\}$.

5.1.2 Preprocessing and Initial Universe Construction: For the movie datasets, we adopt the data pre-processing method of Cocrato and Bridge [6]. For the Amazon dataset, we selected users and items with at least 15 and 80 interactions, respectively, to ensure sufficient training data.

For each dataset, we randomly selected 10,000 test users and constructed for each user an initial universe, where items are associated with a score distribution, generated using CPMF [31], which was shown to achieve state-of-the-art results among algorithms that provide a distribution over the full set of possible scores [6]. CPMF is trained on non-test ratings and is then used to predict score distributions for each test user over candidate items.

We discretize the continuous score distributions produced by CPMF as a preliminary step. All datasets contain explicit feedback over a discrete rating domain. Therefore, continuous distributions are converted into a discrete distribution over the possible score values in *Scores*. Assuming equal spacing between adjacent score values, we define the boundaries between consecutive scores at their midpoints, and extend the first and last regions by half this spacing below the minimum score and above the maximum score, respectively. The probability assigned to a score s_i is then the probability mass of the truncated continuous distribution within the region associated with s_i. The result is a distribution over the score values themselves, which is later used both for rank distribution computation and for feedback incorporation in the adaptive setting.

[2] https://github.com/coralscharf/AdaSeq_UncRec

For every test user, we constructed an initial universe containing selected held-out test items together with sampled negative items, and evaluated recommendation lists of lengths $K \in \{1, \ldots, 20\}$. To reduce variability in the number of held-out test items across users, we set a threshold according to the 80^{th} percentile of the number of held-out test items per user. If a user had at most this number of held-out test items, we included all of them. Otherwise, we randomly sampled this number of held-out test items. Negative items were sampled from items with which the user had not interacted, according to a negative sampling strategy that controls the difficulty of the generated candidate set. The number of sampled negatives was set to 10 times the number of selected held-out test items, with a minimum of 100 negatives and a maximum of 150. The negative sampling strategy was either uniform random sampling or a mixed hard-negative strategy based on the first-stage model. In the mixed hard-negative setting, we ranked the unseen non-test items by their expected score under the first-stage model, formed a hard pool from the top-ranked items, and then sampled a chosen fraction of the negatives from this hard pool, while the remaining negatives were sampled uniformly at random from the rest of the negative pool. The hard level is determined by this fraction, where a larger fraction yields a more difficult candidate set, since it contains more high-scoring negative items that are harder to distinguish from relevant ones.

This construction addresses the sparsity of recommender system data, where test items are rare relative to the full item catalog. In our setting, the initial universe includes held-out test items, so that the adaptive sequential recommendation process can encounter items for which feedback is available and the effect of feedback updates can be meaningfully evaluated.

5.1.3 Recommendation Methods: For each test user, we process a top-K recommendation query over movies (ml-25m and Netflix datasets) or books (Amazon dataset), for recommendation sizes $K \in \{1, 2, \ldots, 20\}$. For quality analysis, we compare static and adaptive sequential recommendation methods. The static semantics are Global Top-k and Expected Score (Section 2.1).

The adaptive framework is evaluated through several variants, obtained by combining different sequential selection semantics with different feedback update mechanisms. The adaptive sequential semantics include three selection semantics, namely A-ES, MPP-1, and A-GT-t (Section 4.2). For feedback update mechanisms, we examine multiple variants based on universe, local, and global updates.

Several approaches require the computation of rank distributions, for which we utilize RankDist with stochastic tie-breaking [24].

5.1.4 Evaluation Measures: In addition to $P@K$ (Section 2.2) we analyze *tail opportunity precision*, which captures the fraction of the remaining achievable gain that is recovered from position k onward. For each starting position $k \in \{1, \ldots, 20\}$, we evaluate the suffix consisting of the items at positions $k, \ldots, k_{\max}$, where $k_{\max} = 20$. The evaluation is based on the reference for $k_{\max} = 20$, while accounting for the part of this reference already consumed by the prefix up to position $k - 1$.

Given a top-$k_{\max}$ answer A and the reference A' for $k_{\max}$, let $A'_{|1:k-1}$ denote the reference remaining after accounting for the prefix $A_{1:k-1}$. Then, the *Tail opportunity precision*, denoted by

TailOpp-P is defined as follows.

$$\text{TailOpp-}P_{k:k_{\max}}(A, A') = \frac{|A_{k:k_{\max}} \cap A'_{|1:k-1}|}{\min\left(|A_{k:k_{\max}}|,\ |A'_{|1:k-1}|\right)}. \quad (13)$$

where $A_{i:j}$ denotes the items in A at positions $i, \ldots, j$.

5.2 Adaptive Sequential vs. Static Recommendation

We first focus on comparing the recommendation lists generated by different ranking semantics, either static or adaptive. Throughout this section, we fix the adaptive update mechanism to a local update configuration (see Section 5.3 for a comparison against no update and global update).

We begin by analyzing the construction of the initial universe, since its difficulty affects both the absolute recommendation quality and the opportunity to incorporate feedback in the adaptive setting.

	$k = 5$		$k = 10$		$k = 20$	
Construction	Static	Adaptive	Static	Adaptive	Static	Adaptive
Random	0.4382	**0.4437**	0.4486	**0.4565**	0.4189	**0.4262**
0.25-hard	0.137	**0.1396**	0.1322	**0.1376**	0.1211	**0.1339**
0.5-hard	0.1356	**0.1375**	0.1281	**0.1312**	0.114	**0.1209**

Table 2: ml-25m comparison of candidate-set constructions using $P@K$ at selected recommendation sizes.

Table 2 compares three initial-universe constructions of different difficulty levels on ml-25m, which we use as a representative dataset for selecting the candidate-set construction. We use Global Top-k as the static method and A-GT-t under the fixed local update configuration as the adaptive method. The random construction yields substantially higher absolute precision values, since the resulting candidate sets are easier. The two hard-negative constructions, with 25% and 50% hard negatives, yield a substantially more challenging recommendation setting, where the 0.25-hard construction achieves higher $P@K$ values. Moreover, the adaptive method outperforms the static method under all three constructions.

We use the 0.25-hard construction in the remainder of the empirical evaluation since it maintains a larger fraction of relevant items within the top-K results, which is important in the adaptive setting for enabling feedback to affect subsequent recommendations.

We next compare the adaptive sequential methods with the static baselines in terms of recommendation quality. Figure 4 shows the average $P@K$ as a function of the recommendation size K for the static and adaptive semantics on ml-25m (left), Netflix (middle), and Amazon (right) datasets. Across all datasets, the rank-based methods A-GT-t, MPP-1, and Global Top-k outperform the score-based methods A-ES and Expected Score. For the ml-25m dataset (Figure 4(a)), A-GT-t outperforms both the other adaptive rank-based method MPP-1 and the static rank-based method Global Top-k, and its advantage becomes more pronounced for $k \geq 5$. MPP-1 shows competitive results compared to Global Top-k for small values of k ($k \leq 5$), whereas Global Top-k outperforms it for larger values. We hypothesize that the weaker performance of MPP-1 relative to Global Top-k stems from its focus on the next most probable item, rather than on membership in the growing top-K prefix. For the Netflix and Amazon datasets (figures 4(b) and 4(c)), both A-GT-t and MPP-1 outperform Global Top-k, although with a smaller margin. Unlike in the ml-25m dataset, here MPP-1 shows performance competitive with A-GT-t. Overall, these results show that A-GT-t is the best performing adaptive semantics in this setting.

Dataset	Tail Start k	Global Top-20	A-GT-t	Improvement (%)
ml-25m	1	0.2057	0.2346	14.04
	5	0.1575	0.2000	26.92
	10	0.1126	0.1670	48.31
	15	0.0794	0.1331	67.56
	20	0.0644	0.1096	70.19
Netflix	1	0.1854	0.2039	9.97
	5	0.1425	0.1652	15.91
	10	0.1085	0.1308	20.57
	15	0.0892	0.1103	23.62
	20	0.0780	0.0987	26.54
Amazon	1	0.0462	0.0497	7.46
	5	0.0325	0.0360	10.85
	10	0.0210	0.0240	14.4
	15	0.0122	0.0144	18.64
	20	0.0084	0.0101	20.25

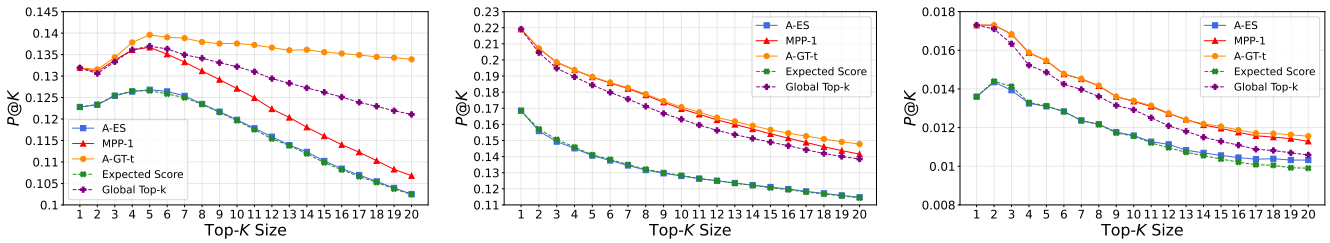
Table 3: Tail opportunity precision comparison between Global Top-20 and A-GT-t.

Next, we focus on the adaptive approach A-GT-t and compare it to the static approach Global Top-k. Table 3 shows TailOpp-P (Section 5.1.4), together with the relative improvement of the adaptive method over the static baseline. The adaptive approach outperforms the static approach by a large margin, and the improvement grows as k increases, indicating that the benefit of adaptation becomes more pronounced further down the recommendation list. This suggests that the adaptive method is especially effective in improving later positions, where the recommendation can benefit from feedback observed earlier in the interaction.

Dataset	K	Tie (%)	List Differs (%)	Win | Non-Tie (%)
ml-25m	5	97.25	24.78	70.55
	10	90.68	73.75	74.68
	15	82.48	93.93	83.11
	20	73.01	98.26	87.51
Netflix	5	94.97	20.40	68.59
	10	90.25	51.00	71.59
	15	85.74	74.60	70.69
	20	78.71	87.79	74.59
Amazon	5	99.61	2.94	82.05
	10	99.21	7.49	70.89
	15	98.61	16.92	81.3
	20	97.87	33.27	83.57

Table 4: User-level A-GT-t vs. Global Top-k under $P@K$.

We also examine the comparison between A-GT-t and Global Top-k at the user level under $P@K$. Table 4 reports, for selected



(a) $P@K$ on ml-25m (b) $P@K$ on Netflix (c) $P@K$ on Amazon

Figure 4: $P@K$ for adaptive and static semantics on the different datasets.

values of K, the percentage of users for whom the two methods tie in $P@K$, the percentage of users for whom the generated top-K lists differ (either in the selected items or in their order), and the conditional win rate of A-GT-t among users for whom the two methods are not tied. For ml-25m and Netflix, the fraction of users for whom the lists differ increases substantially with K, and among users for whom the two methods are not tied, A-GT-t wins in a clear majority of the cases. For Amazon, in contrast, the two methods produce different top-K lists for relatively small fraction of users. Nevertheless, even in these relatively rare non-tied cases, A-GT-t still wins in most of them. Overall, this user-level analysis is consistent with the tail analysis, and further indicates that the adaptive method is particularly effective when it changes the recommendation, especially in later positions of the list.

Dataset	Feedback	Users (%)	Local	No Update	Rel. Diff. (%)
ml-25m	1–2	33.32	0.0715	**0.0723**	-1.12
	3–5	30.77	**0.1853**	0.1842	0.6
	6+	14.85	**0.3572**	0.3502	2.01
Netflix	1–2	30.71	0.0694	**0.07**	-0.74
	3–5	24.72	**0.1811**	0.1783	1.57
	6+	21.66	**0.3773**	0.3559	6.00
Amazon	1–2	15.6	**0.0602**	0.0585	2.9
	3–5	1.34	**0.1616**	0.1187	36.16

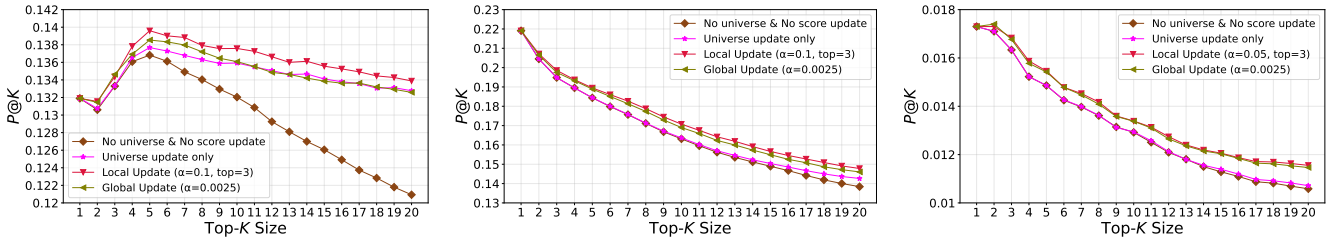
Table 5: Comparing local score update and no score update for A-GT-t under $P@20$, grouped by the number of test items with available feedback that appear in the top-20 answer.

5.3 Analysis of Adaptive Update Methods

We next focus on adaptive sequential recommendation with A-GT-t and analyze the effect of different update methods following user feedback. Figure 5 shows the average $P@K$ as a function of K for four update settings on the ml-25m dataset (left), the Netflix dataset (middle), and the Amazon dataset (right). Three of these settings correspond to the update alternatives introduced in Section 4.3, namely universe update (1) without score update, (2) with local score update, and (3) with global score update. To isolate the contribution of universe reduction, we also consider a baseline in which neither the score distributions nor the universe are updated throughout the interaction.

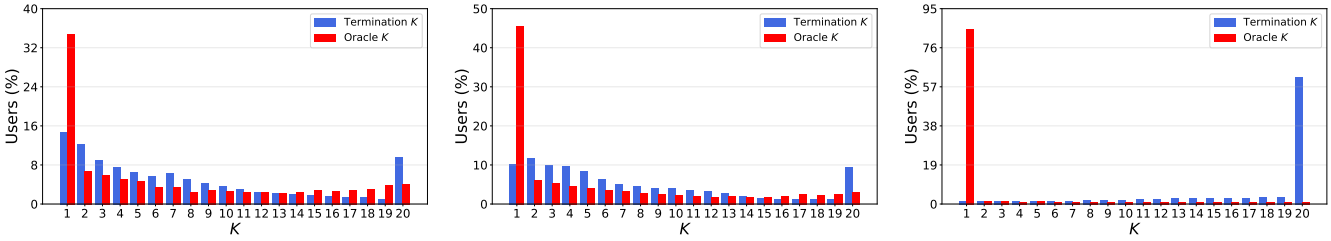
For the ml-25m dataset (Figure 5(a)), the results indicate that universe reduction improves performance relative to the setting in which the universe is not updated (for $K \geq 4$). This is also the largest effect among the design choices considered, and its contribution becomes more pronounced as K increases. Adding local score update on top of universe update yields a further improvement, whereas the global score update variant remains competitive but does not consistently surpass universe update without score update. For the Netflix and Amazon datasets (figures 5(b) and 5(c)), local and global updates outperform the settings without score update. The local update shows a small improvement over the global one on the Netflix dataset, while on the Amazon dataset the two achieve competitive results. In these datasets, universe reduction has a smaller effect, whereas score updating contributes more substantially to the performance improvement.

Table 5 analyzes the effect of local score update compared to no score update (both with universe reduction) by grouping users according to the amount of feedback observed in the top-20 answer. For each group, we report the fraction of users belonging to the group, the precision of the local update and no update variants, and the relative difference between them. Users for whom no feedback items were observed are excluded from the table, since in this case no score update following feedback can be applied, and therefore there is no difference between the two variants. The results show that the benefit of score updating depends on how much feedback is available during the interaction. For ml-25m and Netflix, local score update does not improve performance when only 1–2 feedback items are observed, and may even slightly reduce it. A possible explanation is that such a small amount of feedback is too sparse to support a reliable refinement of the score distributions. However, once more feedback is available, the local update becomes beneficial, and its advantage grows with the number of observed feedback items. On Amazon, local update outperforms the no update variant, although the number of users with observed feedback is much smaller. Notably, users with 3–5 observed feedback items show a large improvement. These results indicate that score updating is most useful when the interaction provides sufficiently rich feedback, whereas very limited feedback may be too sparse to support an effective update.



(a) $P@K$ on ml-25m (b) $P@K$ on Netflix (c) $P@K$ on Amazon

Figure 5: $P@K$ for A-GT-t under different update methods on the different datasets.



(a) ml-25m (b) Netflix (c) Amazon

Figure 6: Comparison between the termination position selected by marginal-item termination and the oracle.

Method	ml-25m	Netflix	Amazon
Static ($K = 20$)	0.1211	0.1385	0.0106
Adaptive w/o Termination ($K = 20$)	0.1339	0.1478	0.0116
Adaptive + Full-Prefix Termination	0.1398	0.148	0.0119
Adaptive + Marginal-Item Termination	**0.2092**	**0.2853**	**0.0363**

Table 6: $P@K$ of the returned list for the termination variants and fixed-length baselines.

5.4 Adaptive Termination Analysis

We now evaluate the termination mechanism used to decide when the sequential recommendation process should terminate (Section 4.4). For the adaptive termination analysis, we use recommendation lists generated by A-GT-t with local score update, and examine the selected termination position and the resulting precision. We compare the two variants of adaptive termination, namely adaptive with full-prefix termination (Eq. 11), and adaptive with marginal-item termination (Eq. 12). These are evaluated against two fixed-length baselines with list size K_{max}: a static recommendation generated by Global Top-K, and an adaptive recommendation generated by A-GT-t without termination.

Table 6 reports the average $P@K$ of the returned list for the three datasets. The termination-based variants improve over both the static baseline and the adaptive baseline without termination across all datasets. In particular, Adaptive + Marginal-Item termination achieves the best results, improving over adaptive without termination by a large margin of 56.24%, 93.03%, and 212.93% on ml-25m, Netflix, and Amazon, respectively. We also observe that adaptive with full-prefix termination improves over adaptive without termination, but only by a small margin. We note that in the full prefix termination the method usually chooses not to terminate, terminating for only a small subset of users. For adaptive with marginal-item termination the average termination point is 7, 8, and 17 for the ml-25m, Netflix, and Amazon datasets, respectively. These results suggest that termination based on the marginal contribution of the next item yields more effective recommendation lengths.

Figure 6 presents a comparison of the distribution over users of the termination position selected by marginal-item termination and the oracle termination position, on the three datasets. Overall, oracle decision tends to be strict, terminating after a single item, which may be too restrictive for users' taste. For ml-25m (left) and Netflix (middle), the marginal-item termination rule provides a varied range of recommendation sizes, which generally decreases with K, especially in the range $K = 2$ to 12. On Amazon, the oracle termination and the marginal-item termination rule show surprising opposite approaches, where the former is almost always set to 1 while the latter is heavily concentrated at $K = 20$. Even with this mismatch, the average precision still improves significantly over choosing a fixed-K value highlighting the benefit of adaptive sequential recommendation through universe and score updates, especially on sparse datasets.

5.5 Efficiency Analysis

We conclude with analysis of computational overhead introduced by adaptive sequential recommendation. To evaluate this overhead, we measure runtime during the actual recommendation execution under the main experimental setup, using A-GT-t with local score

Static Total Time	Adaptive Total Time	Adaptive Time / Step
0.1347	2.5547	0.1277

Table 7: Runtime comparison between static and adaptive sequential recommendation on the ml-25m dataset.

update as the adaptive method and Global Top-k as the static baseline. For the static method, we report the total runtime required to generate the top-$K_{\max}$ answer for $K_{\max} = 20$. For the adaptive method, we report both the total runtime required to generate the full recommendation sequence up to $K_{\max}$ and the average runtime per recommendation step. The latter is particularly relevant in the adaptive setting, since recommendations are generated sequentially.

Table 7 presents the average runtime (seconds) of the static and adaptive methods on the ml-25m dataset, a representative dataset for illustrating the overhead of sequential recommendation relative to static top-K generation. As expected, the adaptive method incurs additional overhead due to sequential generation, repeated rank distribution computation, and update operations after observed feedback. However, although the total adaptive runtime is substantially higher than the static runtime, the adaptive runtime per step is slightly lower than the static total runtime, indicating that adaptive recommendation remains practical in a sequential setting.

6 RELATED WORK

Ranking in recommender systems is traditionally studied in a static top-K setting, where the system estimates a utility score for each candidate item, returning a top-K item list [23]. Several works have incorporated uncertainty into this setting by estimating predicted relevance scores and leveraging score distributions within recommendation tasks [6, 15, 18, 20, 31].

Probabilistic ranking over uncertain data has been extensively studied in the database community [12, 32]. Several semantics and algorithms have been proposed for top-K queries over uncertain data, including [5, 10, 11, 17, 27, 28, 34]. Recent work continues to study efficient ranking and approximating top-K query processing under uncertainty [9]. Recently, [24] studied top-K recommendation under uncertain scores through probabilistic ranking approaches, showing how different ranking semantics align with different recommendation quality objectives. Unlike score-based methods that rely only on score distributions, probabilistic ranking also considers the induced rank distribution of each item and enables rank-based selection methods for recommendation. Our work also uses rank-based methods, and in particular rank distributions, in an adaptive sequential recommendation setting. Specifically, we study an adaptive sequential setting, in which recommendations are generated incrementally and feedback observed during the interaction may affect subsequent selections.

Recent work has also questioned the use of a globally fixed recommendation length and proposed adapting the recommendation size to the user according to a target utility metric [2, 16]. In particular, Kweon et al. [16] propose a framework for selecting the optimal recommendation list size in implicit-feedback settings, where calibrated interaction probabilities are used to choose K by maximizing expected utility. However, their approach is tailored to implicit feedback and selects a cutoff for a static ranking. In contrast, we study a stopping rule that is integrated into an adaptive sequential framework for explicit-feedback settings. Rather than evaluating expected quality solely from predicted uncertainty, we proposed to incorporate observed feedback by using a conditional expected quality measure to determine when to halt the process.

Another line of related work comes from sequential recommendation [8, 13, 30], whose goal is to predict the next item a user will interact with based on a sequence of previous interactions. Sequential recommendation accounts for order and dependencies in the user's interaction history, and typically use the historical interaction sequence as input for next-item prediction. In particular, Fan et al. [8] propose an uncertainty-aware sequential recommendation model in which items and interaction sequences are represented as Gaussian distributions rather than point embeddings, and the next item is predicted by comparing the predicted next-step distribution to candidate item distributions using transformers and Wasserstein distance. Our setting, though sequential in nature, is derived from explicit feedback (rather than implicit) and studies the adaptive construction of a recommendation list under uncertainty.

Reinforcement learning-based methods for interactive recommendation also consider multi-step interaction and adaptation to user responses [1, 35]. However, prior work typically studies recommendation as a sequential decision problem, often with the goal of optimizing long-term cumulative reward. In contrast, our work focuses on uncertainty-aware ranking, studying how observed feedback contributes to generating subsequent recommendations.

Finally, multiple works have studied how recommender systems can be updated when new data arrive, including [21, 22]. For example, Rendle and Schmidt-Thieme [22] propose to retrain the whole user representation, given new ratings arrived. Our work differs in that updates are triggered by a single user's feedback within an ongoing session, and are used to refine score and rank distributions of remaining candidates to improve subsequent selections.

7 CONCLUSIONS

This work introduced the task of adaptive sequential recommendation under uncertain scores, in which recommendations are generated incrementally and user feedback observed during the interaction may affect subsequent selections. We presented a framework that, given uncertain explicit scores, generates the next recommendation at each step and updates the recommendation model based on the observed recommendation outcome and, when available, user feedback. In addition, we proposed a termination rule for determining the recommendation size with the goal of optimizing precision. We conducted an empirical evaluation on multiple datasets, demonstrating the advantage of adaptive sequential methods over static methods, and investigated the impact of different update strategies and termination rules.

By moving from a static recommendation paradigm to an adaptive sequential one, this work opens several directions for future research. In particular, we plan to investigate hybrid methods that combine static and sequential approaches, as well as additional update mechanisms for recommendation models under uncertain scores that incorporate user feedback. We also intend to study termination criteria for additional quality measures beyond precision.

REFERENCES

[1] M Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–38.

[2] Nitin Bisht, Xiuwen Gong, and Guandong Xu. [n.d.]. Guaranteed Top-Adaptive-K in Recommendation. ([n. d.]).

[3] Jesús Bobadilla, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez. 2013. Recommender systems survey. *Knowledge-based systems* 46 (2013), 109–132.

[4] Eleonora Ciceri, Piero Fraternali, Davide Martinenghi, and Marco Tagliasacchi. 2015. Crowdsourcing for top-k query processing over uncertain data. *IEEE Transactions on Knowledge and Data Engineering* 28, 1 (2015), 41–53.

[5] Graham Cormode, Feifei Li, and Ke Yi. 2009. Semantics of ranking queries for probabilistic data and expected ranks. In *2009 IEEE 25th International Conference on Data Engineering*. IEEE, 305–316.

[6] Victor Coscrato and Derek Bridge. 2023. Estimating and Evaluating the Uncertainty of Rating Predictions and Top-n Recommendations in Recommender Systems. *ACM Transactions on Recommender Systems* (2023).

[7] Osnat Drien, Matanya Freiman, Antoine Amarilli, and Yael Amsterdamer. 2023. Query-guided resolution in uncertain databases. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.

[8] Ziwei Fan, Zhiwei Liu, Shen Wang, Lei Zheng, and Philip S Yu. 2021. Modeling sequences as distributions with uncertainty for sequential recommendation. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 3019–3023.

[9] Su Feng, Boris Glavic, and Oliver Kennedy. 2023. Efficient Approximation of Certain and Possible Answers for Ranking and Window Queries over Uncertain Data. *Proceedings of the VLDB Endowment* 16, 6 (2023).

[10] Tingjian Ge, Stan Zdonik, and Samuel Madden. 2009. Top-k queries on uncertain data: on score distribution and typical answers. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 375–388.

[11] Ming Hua, Jian Pei, Wenjie Zhang, and Xuemin Lin. 2008. Ranking queries on uncertain data: a probabilistic threshold approach. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 673–686.

[12] Ihab F Ilyas and Mohamed A Soliman. 2011. Probabilistic ranking techniques in relational databases. *Synthesis Lectures on Data Management* 3, 1 (2011), 1–71.

[13] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[14] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

[15] Yehuda Koren and Joe Sill. 2011. Ordrec: an ordinal model for predicting personalized item rating distributions. In *Proceedings of the fifth ACM conference on Recommender systems*. 117–124.

[16] Wonbin Kweon, SeongKu Kang, Sanghwan Jang, and Hwanjo Yu. 2024. Top-Personalized-K Recommendation. In *Proceedings of the ACM Web Conference 2024*. 3388–3399.

[17] Jian Li, Barna Saha, and Amol Deshpande. 2011. A unified approach to ranking in probabilistic databases. *The VLDB Journal* 20 (2011), 249–275.

[18] Andriy Mnih and Russ R Salakhutdinov. 2008. Probabilistic matrix factorization. In *Advances in neural information processing systems*. 1257–1264.

[19] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 188–197.

[20] Fernando Ortega, Raúl Lara-Cabrera, Ángel González-Prieto, and Jesús Bobadilla. 2021. Providing reliability in recommender systems through Bernoulli matrix factorization. *Information sciences* 553 (2021), 110–128.

[21] Danni Peng, Sinno Jialin Pan, Jie Zhang, and Anxiang Zeng. 2021. Learning an adaptive meta model-generator for incrementally updating recommender systems. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 411–421.

[22] Steffen Rendle and Lars Schmidt-Thieme. 2008. Online-updating regularized kernel matrix factorization models for large-scale recommender systems. In *Proceedings of the 2008 ACM conference on Recommender systems*. 251–258.

[23] Francesco Ricci, Lior Rokach, and Bracha Shapira. 2021. Recommender systems: Techniques, applications, and challenges. *Recommender systems handbook* (2021), 1–35.

[24] Coral Scharf, Carmel Domshlak, Avigdor Gal, and Haggai Roitman. 2025. A Rank-Based Approach to Recommender System's Top-K Queries with Uncertain Scores. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.

[25] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. Autorec: Autoencoders meet collaborative filtering. In *Proceedings of the 24th international conference on World Wide Web*. 111–112.

[26] Roee Shraga and Avigdor Gal. 2022. PoWareMatch: A quality-aware deep learning approach to improve human schema matching. *ACM Journal of Data and Information Quality (JDIQ)* 14, 3 (2022), 1–27.

[27] Mohamed A Soliman and Ihab F Ilyas. 2009. Ranking with uncertain scores. In *2009 IEEE 25th International Conference on Data Engineering*. IEEE, 317–328.

[28] Mohamed A Soliman, Ihab F Ilyas, and Kevin Chen-Chuan Chang. 2007. Top-k query processing in uncertain databases. In *2007 IEEE 23rd International Conference on Data Engineering*. IEEE, 896–905.

[29] Dan Suciu, Dan Olteanu, Christopher Ré, and Christoph Koch. 2022. *Probabilistic databases*. Springer Nature.

[30] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[31] Chao Wang, Qi Liu, Runze Wu, Enhong Chen, Chuanren Liu, Xunpeng Huang, and Zhenya Huang. 2018. Confidence-aware matrix factorization for recommender systems. In *Proceedings of the AAAI Conference on artificial intelligence*, Vol. 32.

[32] Yijie Wang, Xiaoyong Li, Xiaoling Li, and Yuan Wang. 2013. A survey of queries over uncertain data. *Knowledge and information systems* 37, 3 (2013), 485–530.

[33] Chen Jason Zhang, Lei Chen, Hosagrahar V Jagadish, and Chen Caleb Cao. 2013. Reducing uncertainty of schema matching via crowdsourcing. *Proceedings of the VLDB Endowment* 6, 9 (2013), 757–768.

[34] Xi Zhang and Jan Chomicki. 2009. Semantics and evaluation of top-k queries in probabilistic databases. *Distributed and parallel databases* 26, 1 (2009), 67–126.

[35] Lixin Zou, Long Xia, Pan Du, Zhuo Zhang, Ting Bai, Weidong Liu, Jian-Yun Nie, and Dawei Yin. 2020. Pseudo Dyna-Q: A reinforcement learning framework for interactive recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 816–824.